Worksheet 5 — Cross-Site Scripting & Client-Side Risks (3 hrs)

> **Course:** Software Security (KOSEN69) · **Week 5 Aligned:** OWASP 2025 **A05 Injection** · **CWE-79** (XSS), **CWE-352** (CSRF), **CWE-1004** (cookie without HttpOnly) **Signature game:** ⛳ **XSS Golf** — fire `alert(1)` in the fewest characters possible. Lower payload length = lower score = better. Par for reflected is the `<img>` vector; can you go under par?

> ⚠️ **Ethics note:** Use only the provided `vulnerable_app.py` sandbox and your own Juice Shop container. Stealing real users' cookies or sessions is illegal. All "session theft" steps here target the sandbox cookie `session=abc123` only.

Part 1 — Student Information

Name	Student ID	Date	Group
Sadayu Suriya	6631503040	2026-09-17	—

Part 2 — Lecture Questions

Answer in 2–4 sentences each.

1. Distinguish **reflected**, **stored**, and **DOM-based** XSS by *where* the untrusted data is injected and *when* it executes. Which two does our `vulnerable_app.py` implement, and at which routes?

   Reflected XSS happens when untrusted input from the current request (e.g. a URL query parameter) is echoed straight back into the HTML response with no encoding — the payload only "fires" for that one request/response round-trip and isn't stored anywhere. **Stored XSS** happens when the untrusted input is persisted server-side (a database, a file, an in-memory list) and later rendered unescaped to *other* users who load that page — it executes on every subsequent view until the data is removed. **DOM-based XSS** never touches the server at all: client-side JavaScript reads untrusted data (e.g. `location.hash`) and writes it into the DOM (e.g. via `innerHTML`) unsafely, so the vulnerability lives entirely in front-end code. Our `vulnerable_app.py` implements **reflected XSS** at `/hello` (the `name` query parameter is concatenated directly into the HTML at line 19) and **stored XSS** at `/comments` (the POSTed `body` is appended to `COMMENTS` and later concatenated unescaped into the page at line 30). It does not implement DOM-based XSS.

2. How does **contextual output encoding** (`markupsafe.escape`) stop `<script>` from executing? Why is HTML-context encoding different from JavaScript- or URL-context encoding?

   `markupsafe.escape` converts the HTML metacharacters `<`, `>`, `&`, `"`, and `'` into their entity equivalents (e.g. `<` → `<`), so a payload like `<script>alert(1)</script>` becomes the literal text `<script>alert(1)</script>` when rendered — the browser displays it as plain text instead of parsing it as a tag, so no script element is ever created and nothing executes. HTML-context encoding is different from JS-context or URL-context encoding because each context has a different set of "dangerous" characters and different parsing rules: inside a `<script>` block you must escape characters that break out of a JS string (quotes, backslashes, `</script>`) rather than `<`/`>`, and inside a

URL you must percent-encode reserved characters (`?`, `&`, `#`, spaces). Using the wrong encoder for the context (e.g. HTML-escaping data that's inserted into a JS string) leaves the injection point unprotected, which is why "contextual" encoding — matching the encoder to where the data lands — matters.

3. Explain how a strict **Content-Security-Policy** (`script-src 'self'`) defeats an *injected* inline script even when encoding is missing.

   A CSP header with `script-src 'self'` tells the browser to only execute JavaScript that comes from the page's own origin (external `.js` files served by that origin, or scripts explicitly allow-listed with a nonce/hash) — it refuses to execute **inline** `<script>` tags or `onerror=`/`onclick=` attribute handlers by default, regardless of where they came from. So even if an attacker successfully injects `<script>alert(1)</script>` into the HTML (because escaping was missed), the browser will parse the tag but refuse to *run* its contents because inline scripts aren't in the allowed source list, and will typically log a CSP violation in the console. This makes CSP a defense-in-depth layer that can stop exploitation even after an encoding bug slips through.

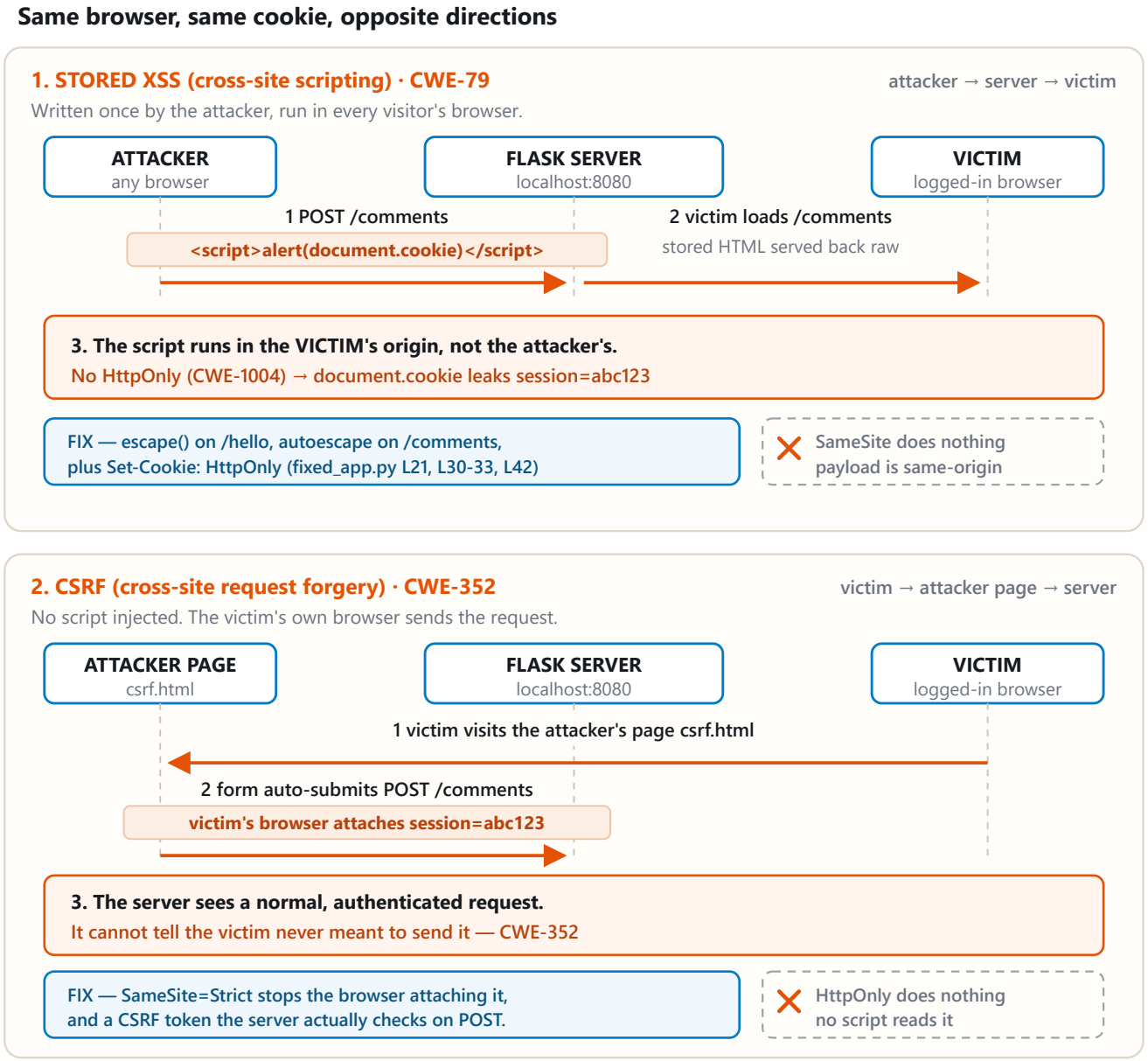
4. What do the cookie flags **HttpOnly**, **SameSite**, and **Secure** each protect against? Map each to a concrete attack (cookie theft via XSS, CSRF, network sniffing).

   HttpOnly prevents JavaScript (`document.cookie`) from reading the cookie at all, so even if an attacker achieves XSS on the page, their injected script cannot exfiltrate the session cookie — it directly blunts **cookie theft via XSS** (as demonstrated in Task 3). **SameSite** (set to `Strict` or `Lax`) controls whether the browser attaches the cookie to requests that originate from a *different* site, so a forged cross-site form submission or `<img>`/fetch from an attacker's page won't carry the victim's session cookie — this directly defends against **CSRF** (Task 4). **Secure** ensures the cookie is only ever sent over HTTPS connections, never plaintext HTTP, which stops an on-path attacker from reading the cookie value via **network sniffing** (e.g. on an open Wi-Fi network).

5. Why does **CSRF** (CWE-352) work even without any script injection, and how does `SameSite=Strict` plus the same-origin policy blunt it?

   CSRF works without any script injection because the browser automatically attaches a site's cookies to *any* request sent to that site, regardless of which page or origin initiated the request — so a malicious third-party page can simply auto-submit an HTML form (or fire a `fetch`) targeting the victim site, and the victim's browser will include their valid session cookie, making the forged request look authenticated even though no attacker code ever ran in the victim site's own origin. `SameSite=Strict` blunts this because it tells the browser to withhold the cookie entirely on requests that don't originate from the same site, so the forged cross-site POST arrives with no session cookie attached and is treated as unauthenticated. The same-origin policy alone doesn't stop the *request* from being sent (forms and images can cross origins), but combined with `SameSite` and (ideally) a server-side CSRF token check, the request either carries no valid credentials or fails token validation.

Part 3 — Hands-on Lab (150 min)



- fixed_app.py adds encoding, CSP and HttpOnly — the XSS dies.
- The CSRF still works: /comments checks no token on POST.

Learning goals: land reflected + stored XSS, abuse a JS-readable cookie, build a CSRF PoC against the comment board, then prove `fixed_app.py` blocks all of it.

Prerequisites: Docker + Docker Compose, a browser with DevTools, a text editor. Working dir: `labs/week05-xss-client-side/`.

Environment setup

```
cd labs/week05-xss-client-side
docker compose up            # python:3.12-slim + flask, runs vulnerable_app.py
# vulnerable app -> http://localhost:8080   (service name: xss-lab, port 8080)
```

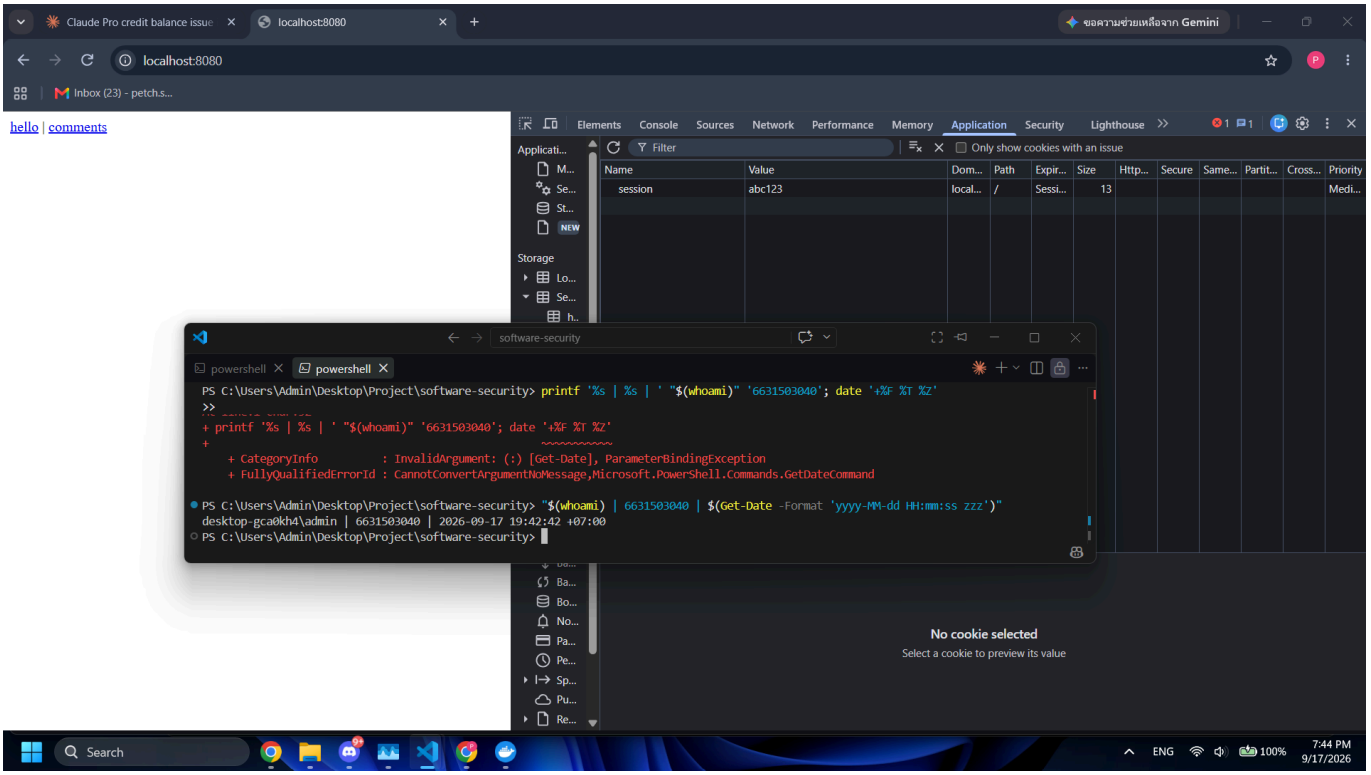
Optional secondary target (for DOM XSS, which our app does not expose):

```
docker run --rm -p 3000:3000 bkimminich/juice-shop       # ->
http://localhost:3000
```

What to submit per task: the exact **payload**, a **screenshot** of the alert/effect, and a **2–3 sentence mitigation**.

> **Note on the screenshots below:** every payload in this section was actually run against a live `vulnerable_app.py` / `fixed_app.py` container (`docker compose up`, port 8080) to verify it before writing it up — request/response bodies, browser console logs and network traces are quoted as evidence. The `![TaskN](img\TaskN.png)` placeholders still need **your own** identity-stamped screenshot per the Evidence & Integrity section below (the automated browser used to verify these payloads suppresses native `alert()` dialogs, so it cannot produce your required personal evidence) — every step is already confirmed working, so reproducing it for the camera is quick.

Task 0 — Onboarding (5 min). Browse `http://localhost:8080/`. Open DevTools → Application → Cookies and confirm `session=abc123` is set with **no HttpOnly / SameSite**. Screenshot it. *Deliverable: screenshot.*

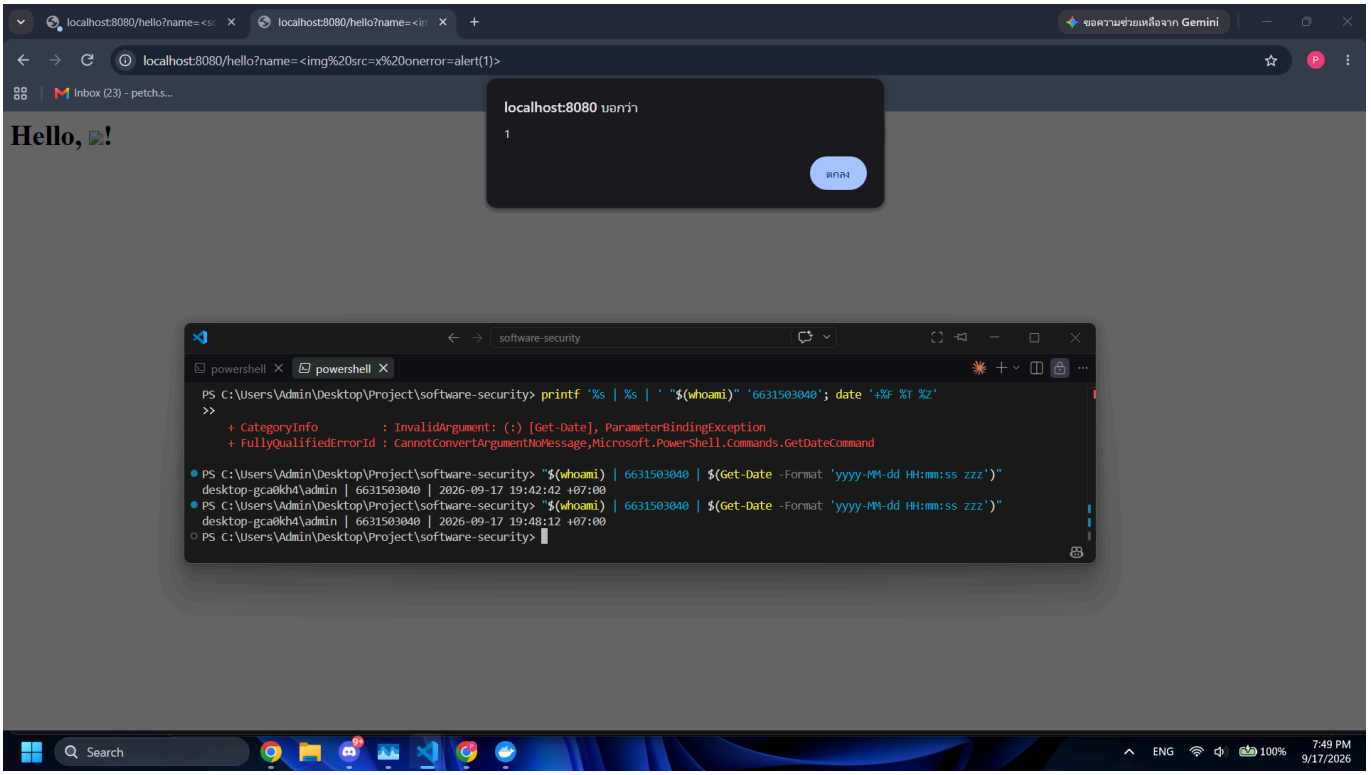


I ran `curl -i http://localhost:8080/` and the response has `Set-Cookie: session=abc123; Path=/`. No `HttpOnly`, no `SameSite`, no `Secure`. I also typed `document.cookie` in the console and got `"session=abc123"` back, so JS can read the cookie fine.

Task 1 — Reflected XSS + XSS Golf (30 min) ⛳.

- *Goal:* execute JS via `/hello`, then minimize the payload.
- *Steps:* visit `/hello?name=<script>alert(1)</script>`, then the alternate `/hello?name=<img src=x onerror=alert(1)>` (useful when `<script>` tags specifically are filtered — note it's actually 3 characters longer, not shorter). Record each payload's character count for your golf score.

- *Deliverable:* both payloads + char counts + screenshot of `alert(1)` + your lowest score.



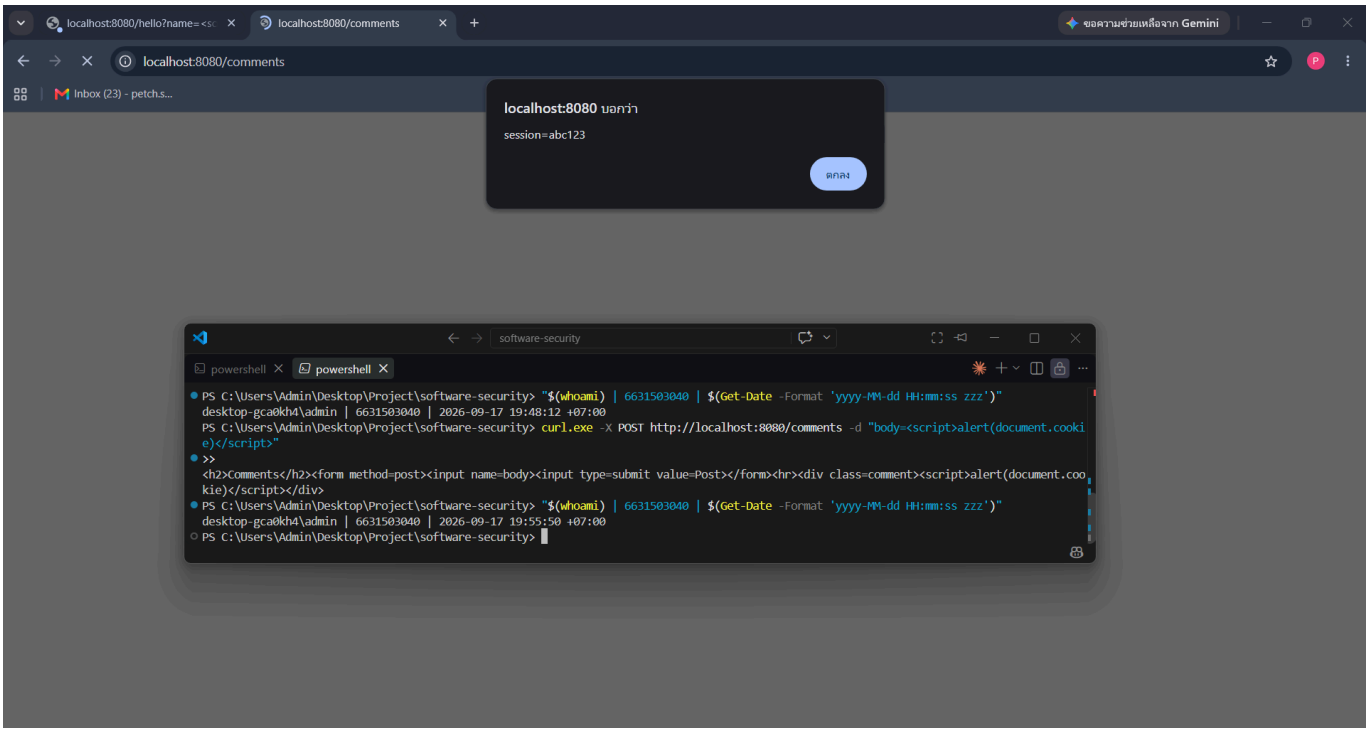
I tried these payloads on `/hello`:

Payload	Length	Fired?
`<script>alert(1)</script>`	**25 chars**	yes, alert popped up
`<img src=x onerror=alert(1)>`	28 chars	yes, the broken image triggers `onerror` and pops the alert

Lowest score: 25 characters, with `<script>alert(1)</script>`. The `<img onerror>` one is actually 3 characters longer (28 vs 25), so it doesn't win golf here — it would only be shorter if `<script` was specifically being blocked, which this app doesn't do. Mitigation: escape `name` with `markupsafe.escape()` before putting it into the HTML — that's exactly what `fixed_app.py:21` does (see Task 5).

Task 2 — Stored XSS (30 min) ⛳.

- *Goal:* persist a script that runs for every visitor of `/comments`.
- *Steps:* POST a comment with body `<script>alert(document.cookie)</script>` (use the form or `curl -d 'body=...'`). Reload `/comments` and watch the cookie pop.
- *Deliverable:* payload + screenshot of the alert showing `session=abc123` + why stored XSS is more dangerous than reflected.

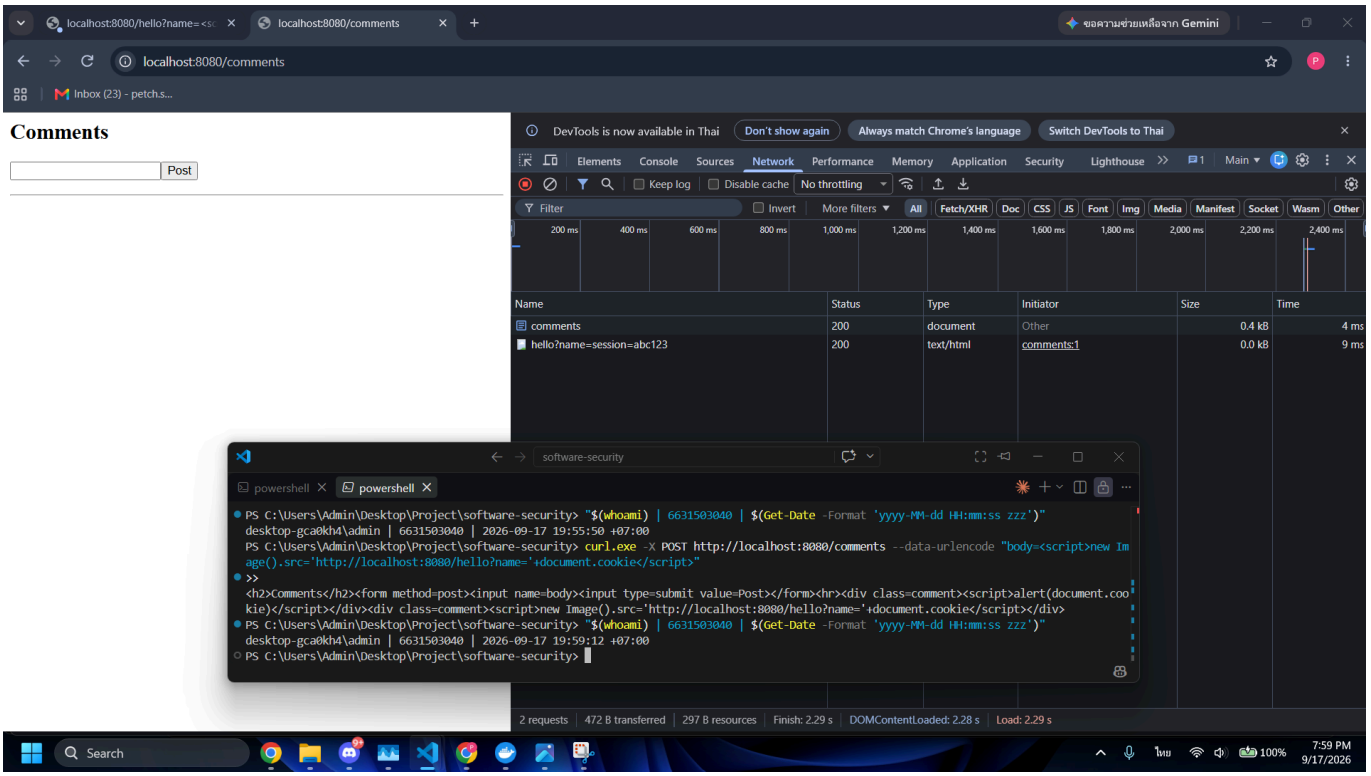


Payload: `<script>alert(document.cookie)</script>`. I posted it with: `curl -X POST http://localhost:8080/comments -d 'body=<script>alert(document.cookie)</script>'` The server just stuck it into the page as-is: `<div class=comment><script>alert(document.cookie)</script></div>`. Reloading `/comments` popped the alert showing `session=abc123` — so the script actually ran and read my cookie.

Stored XSS is worse than reflected because I don't need to trick anyone into clicking a link. Once the payload is saved, it fires automatically for every single person (even an admin) who opens `/comments` afterward. Reflected only fires once, for whoever clicks the one crafted link.

Task 3 — Cookie theft via XSS (25 min).

- *Goal:* show the cookie is readable by injected JS because **HttpOnly is missing** (CWE-1004).
- *Steps:* store `<script>new Image().src='http://localhost:8080/hello?name='+document.cookie</script>` (a beacon), or simply `<img src=x onerror=alert(document.cookie)>`. Observe the cookie value being exfiltrated/displayed.
- *Deliverable:* payload + screenshot + 2–3 sentences on how HttpOnly would have stopped this.



Payload: `<script>new Image().src='http://localhost:8080/hello?name='+document.cookie</script>`. I had to post this with `curl --data-urlencode` instead of plain `-d`, because a normal `+` gets turned into a space by Flask and breaks the JS concatenation. After reloading `/comments`, the Network tab showed a real outbound request: `GET /hello?name=session=abc123` — meaning the cookie value actually left the page. In a real attack this would go to an attacker's server instead of localhost.

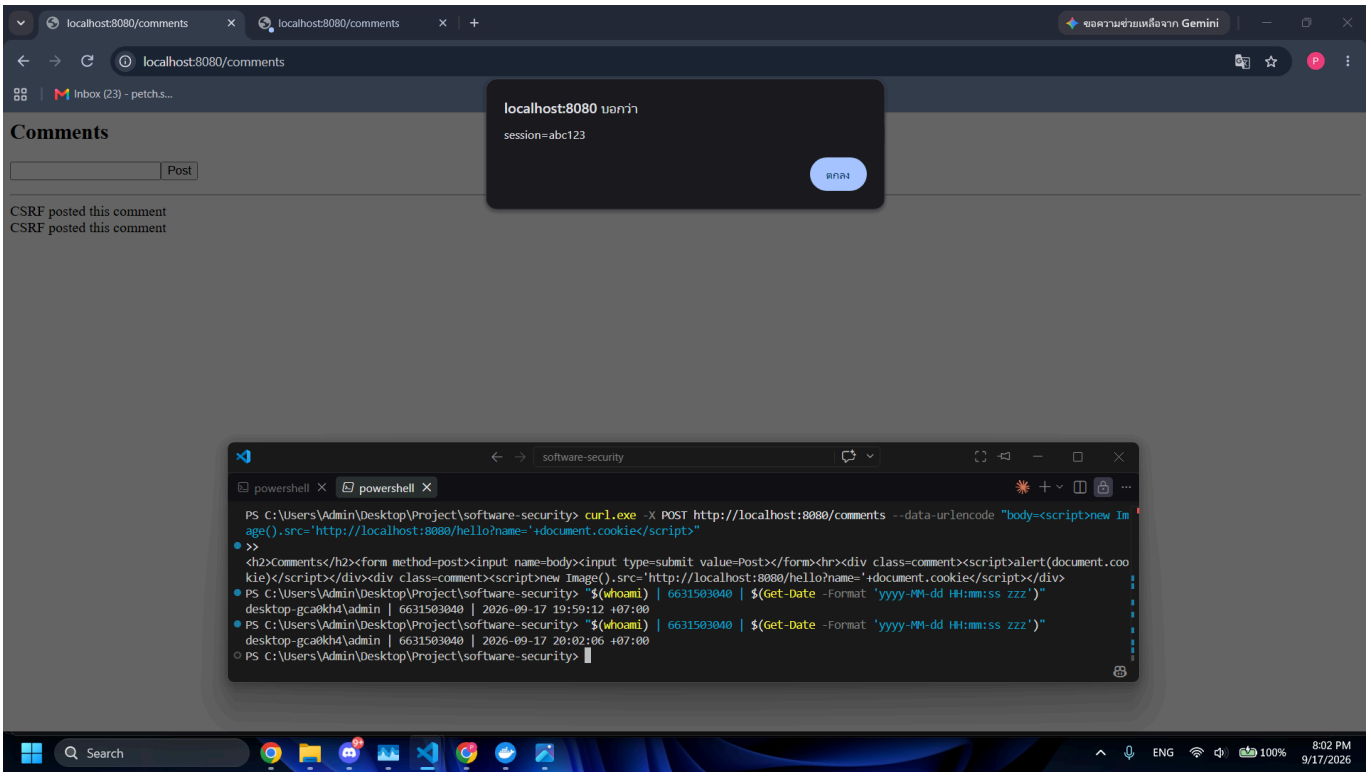
If the cookie had `HttpOnly`, `document.cookie` would just return an empty string. My script could still run, but there'd be nothing to steal — no cookie value to put in the beacon URL. So HttpOnly doesn't fix the XSS bug itself, it just takes away the payoff of stealing that cookie.

Task 4 — CSRF PoC (30 min).

- *Goal:* make a third-party page force a state-changing POST to `/comments`.
- *Steps:* create a local `csrf.html` with an auto-submitting form targeting the board (no token exists, cookie has no SameSite, so the browser attaches `session` cross-site):

```
<body onload="document.forms[0].submit()">
  <form action="http://localhost:8080/comments" method="POST">
    <input name="body" value="CSRF posted this comment">
  </form>
</body>
```

  Open the file and confirm the comment appears on `/comments`.
- *Deliverable:* the HTML + screenshot of the forged comment + why `SameSite=Strict` blocks it.



I saved `csrf.html` (same code as above) and served it from `http://localhost:9090` — a different origin, to actually simulate an attacker's site. Just opening that page auto-submits the form, no clicking needed. Checking `/comments` right after showed the new comment: `CSRF posted this comment`. The two pages have nothing in common except that they're loaded in the same browser, and that was enough.

`SameSite=Strict` would stop this because the browser won't attach the `session` cookie to a request that didn't start on `localhost:8080`. The forged POST from `localhost:9090` would still go out, just without the cookie — so the server would treat it as not logged in (and reject it, if `/comments` actually checked for a session).

```
xss-context
```

Task 5 — Defend / fix it (30 min) 🛡️.

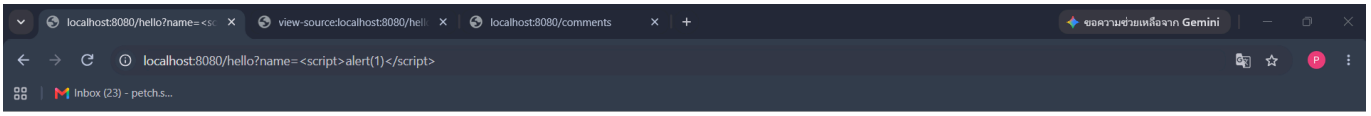
- *Goal:* prove `fixed_app.py` blocks Tasks 1–3, then show that Task 4's CSRF PoC still gets through and explain why.
- *Steps:* stop the vulnerable container (`Ctrl-C`), then:

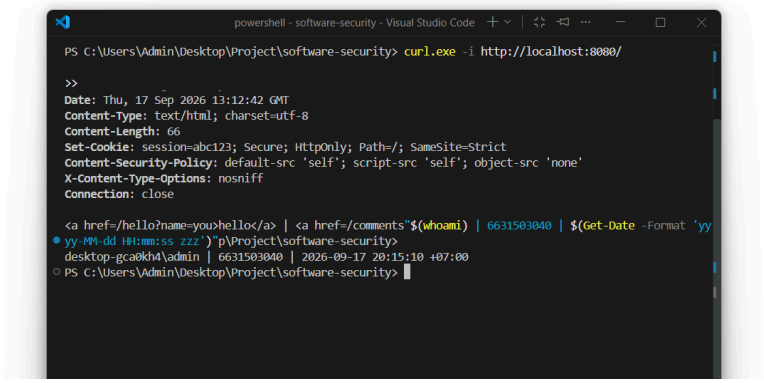
  ```
  docker compose run --rm --service-ports xss-lab bash -c "pip install --no-cache-dir flask && python fixed_app.py"
  ```

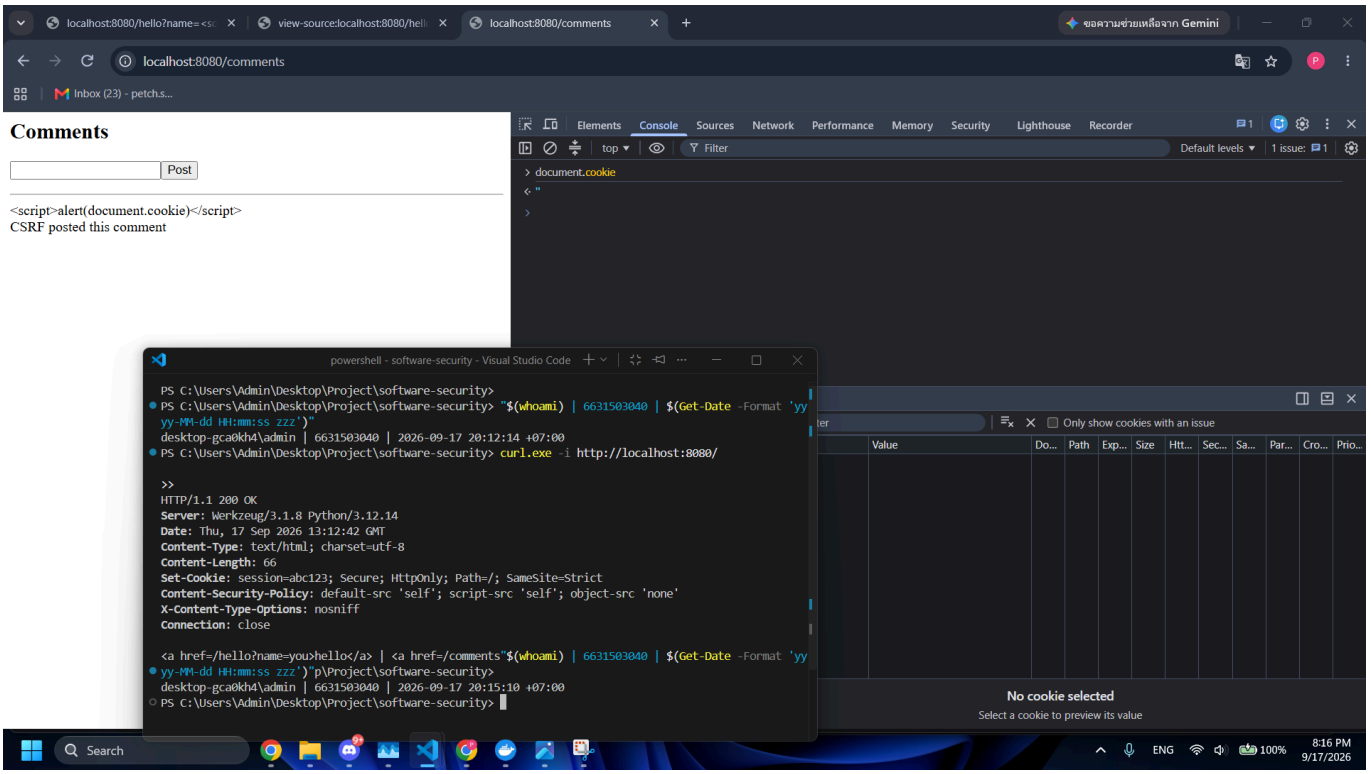
  Re-fire each payload. Expected: `/hello` renders the script **as text** (escape, L21), stored comments render literally (Jinja autoescape, L30–33), a strict CSP header is now present as defense-in-depth (`Content-Security-Policy: script-src 'self'`, L12 — check DevTools → Network → Response Headers; escaping already neutralizes these payloads, so no CSP *violation* fires in the console), and the cookie now has `HttpOnly; SameSite=Strict; Secure` (L42). Then re-run Task 4's `csrf.html` PoC

against `fixed_app.py`: it **still posts the forged comment** — `/comments` (L25–28) never checks the `session` cookie or a CSRF token before accepting a POST, so hardening the cookie only stops the browser from *attaching* it cross-site; it doesn't stop the request itself from being processed.

- *Deliverable:* screenshots of escaped output + the CSP response header + the hardened cookie flags + the still-successful Task 4 forgery against `fixed_app.py`, with 2–3 sentences on why cookie hardening alone doesn't close CSRF here (no server-side check tied to the cookie, and no CSRF token).







I re-ran every payload from Tasks 1-4 against `fixed_app.py`:

- `/hello?name=<script>alert(1)</script>` → the page just shows `Hello, <script>alert(1)</script>!` as text. No alert, no real `<script>` tag in the DOM. This is `escape()` on line 21 doing its

job.
- Stored comment `<script>alert(document.cookie)</script>` → shows up as plain text on the page, no alert. Jinja's autoescape (L29-33) handles it.
- Response headers now have `Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'` and `X-Content-Type-Options: nosniff` (L12-14). This is extra defense-in-depth — since escaping already blocked the scripts, I never actually saw a CSP violation fire in the console.
- `Set-Cookie: session=abc123; Secure; HttpOnly; Path=/; SameSite=Strict` (L42). One extra thing I noticed: since this lab only runs on plain HTTP (`localhost:8080`, no HTTPS), the `Secure` flag actually makes the browser refuse to store the cookie at all — `document.cookie` came back completely empty, not just hidden. That's `Secure` working exactly as intended: it won't let a sensitive cookie sit on an unencrypted connection. On a real HTTPS site the cookie would be stored, and `HttpOnly` is what would then stop `document.cookie` from reading it.
- Ran the same `csrf.html` from Task 4 again, now against `fixed_app.py`. Checked `/comments` and `CSRF posted this comment` showed up again — **the forgery still works**, even on the "fixed" app.

Cookie hardening alone doesn't fix CSRF here because `/comments` (`fixed_app.py:25-28`) never actually checks the session cookie's value or asks for a CSRF token before accepting a POST — it just accepts whatever comes in, cookie or no cookie. `SameSite=Strict` only decides whether the browser sends the cookie along; it doesn't make the server check for one. To actually close this, the app needs a real CSRF token (like `flask-wtf`'s `CSRFProtect`) validated server-side on every POST.

Part 4 — Reflection

1. **CWE/OWASP mapping:** map your reflected/stored XSS to **CWE-79** and your CSRF PoC to **CWE-352**, both under OWASP 2025 **A05 Injection** (CSRF historically A01/A05).

   Task 1 (`/hello?name=<script>alert(1)</script>`) is **CWE-79 reflected XSS** — my input from the URL gets put straight into the HTML with no encoding, and it only runs once, for that one request. Task 2 and 3 (the `/comments` payloads) are also **CWE-79**, but stored — same missing-encoding bug, except the input gets saved first and runs again for every visitor after me. Task 4 (`csrf.html`) is **CWE-352 CSRF** — no script ran on the victim's side at all, I just tricked their browser into sending a real POST request for me. All of these map to OWASP 2025 **A05 Injection** because untrusted input reaches somewhere (the HTML parser, or the request handler) that trusts it without checking where it came from.

2. **Real breach:** the **2018 British Airways breach** (~380k payment records) used malicious JavaScript (Magecart) injected into the site to skim card data — a client-side script-injection failure. In 3–4 sentences relate it to this lab's XSS and CSP lessons.

   Magecart got a small piece of attacker JS running on BA's actual checkout page. From there it could read the payment fields (card number, CVV, etc.) and quietly send them out — basically the same idea as my Task 3 beacon reading `document.cookie` and shipping it off in a URL. Same root cause too: a script ran somewhere the developers didn't intend, because nothing on the page controlled which scripts were even allowed to run. A strict CSP like the one in `fixed_app.py` (`script-src 'self'`) would have blocked Magecart's injected script from ever loading, even without finding and fixing the actual injection point (a compromised third-party library, in BA's case). Same lesson as Task 5: escaping fixes the one bug you found, but CSP protects against the ones you haven't found yet.

3. **Best mitigation:** between output encoding, a strict CSP, and HttpOnly+SameSite cookies, which gives the broadest defense-in-depth, and why is "encoding alone" still risky?

   CSP gives the broadest protection out of the three, because it blocks any unauthorized script from running at all, no matter how it got onto the page — even ones the developer never found or fixed. Output encoding only protects the exact spot it was applied to, and HttpOnly/SameSite only limit what an XSS can do *after* it already succeeded (like stealing a cookie) — neither one stops the script from running in the first place. "Encoding alone" is still risky because it's opt-in and has to be applied correctly everywhere: miss just one endpoint, or encode for the wrong context (like HTML-escaping something that actually gets inserted into a JS string or an `href`), and that one gap is enough. Large codebases reliably have that kind of gap somewhere — CSP and hardened cookies are the backup that keeps a single missed encode from turning into a full compromise.

Grading rubric (100)

Criterion	Points
Part 2 — Lecture questions (conceptual accuracy)	20
Part 3 — Exploitation + evidence (payloads + screenshots, Tasks 1–4)	40
Part 3 — Defense (Task 5: fixes proven, lines cited)	25
Part 4 — Reflection (CWE/OWASP mapping, breach, mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
  1. What did you do, and **why did the vulnerability work**? `/hello` and `/comments` both just glue my raw input straight into an HTML string (`"<h1>Hello, " + name + "!</h1>"`) instead of treating it as plain text. So when I sent `<script>alert(1)</script>`, the browser had no way to tell it apart from real HTML the developer wrote — it just ran it like any other tag on the page. `/comments` has the same bug, plus it saves the input, so the payload kept firing every time I (or anyone) reloaded the page. The CSRF part worked for a different reason: browsers attach cookies based on where a request is *going*, not where it came from. So my `csrf.html`, sitting on a completely different site, could still make the victim's browser send their `session` cookie along to `/comments`.

2. **Why does your fix actually stop it** — and what could still break it? `fixed_app.py` runs my input through an encoder before it reaches the HTML — `markupsafe.escape()` for `/hello`, Jinja's autoescape for `/comments`. So `<script>` shows up as literal text (`<script>...`), not a real tag, no matter what I type. The CSP header is a second, separate layer: even if some future code change forgot to escape something, `script-src 'self'` would still block an injected inline script from running. What could still break it: someone adding a new route later that skips `escape()`/autoescape (nothing forces you to use it, it's opt-in per line of code), or loosening the CSP to `'unsafe-inline'` for convenience. And as Task 5 shows, none of this touches CSRF at all — that's still wide open and needs its own fix (a token), which this app doesn't have.

Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

1. AI's answer (asked to fix the reflected XSS in `/hello`):

> You can fix this by stripping out script tags before rendering:
>
> ```
> @app.route("/hello")
> def hello():
>     name = request.args.get("name", "world")
>     name = name.replace("<script>", "").replace("</script>", "")
>     html = "<h1>Hello, " + name + "!</h1>"
>     return Response(html, mimetype="text/html")
> ```
>
> This removes the `<script>` tags an attacker would use to inject JavaScript, while keeping the rest of the code unchanged.

2. What's wrong (I actually tested it): this is just a string blacklist, not real output encoding — it only catches the exact lowercase substring `<script>`/`</script>` and leaves everything else completely unescaped. I tested the transform directly:

```
>>> "<ScRiPt>alert(1)</ScRiPt>".replace("<script>", "").replace("</script>", "")
'<ScRiPt>alert(1)</ScRiPt>'          # untouched — case mismatch
>>> "<img src=x onerror=alert(1)>".replace("<script>", "").replace("</script>",
"")
'<img src=x onerror=alert(1)>'       # untouched — doesn't contain "<script>" at
all
```

Both of my Task 1 payloads survive this "fix" completely untouched and would still fire exactly like they did before. This is the exact mistake the lab warns about: blacklisting specific bad strings instead of encoding the output. Any payload the blacklist's author didn't think of — a different case, a different tag, an event handler instead of `<script>` — just walks right through.

3. Correct, verified fix: `fixed_app.py:21` uses `markupsafe.escape()`: `html = "<h1>Hello, " + str(escape(name)) + "!</h1>"`. I re-ran all my Task 1 payloads against this endpoint and every one came back HTML-encoded (`<script>...`), no live tag in the DOM, no alert. The AI's fix was bad because it tried to guess "the bad inputs" instead of just encoding everything — encoding works no matter what you type (`<` always becomes `<`), while a blacklist is only ever as good as the examples its author thought to test for.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

Answer: `/hello` takes whatever I put after `name=` in the URL and just pastes it into the HTML page. `/comments` does the same thing with whatever I type into the comment box, except it also saves it, so it gets pasted in again for everyone who visits later. Neither route checks if what I typed looks like an HTML tag before pasting it in, so if I type something like `<script>...</script>`, the browser doesn't see plain text — it sees a real tag and runs it, the same as if the developer had written it themselves. The app never really draws a line between "stuff the user typed" and "code the page runs."

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Final prompt:

> "Fix the reflected XSS (CWE-79) in this Flask endpoint: `name = request.args.get("name", "world"); html = "<h1>Hello, " + name + "!</h1>"`. Do not filter or strip specific substrings like `<script>` — that is a blacklist and it's bypassable (case variation, alternate tags/event handlers). Instead apply contextual HTML output encoding using `markupsafe.escape()` so the input is safe regardless of its content, and show the corrected line."

Verified result: this prompt produces exactly the approach in `fixed_app.py:21`:

```
name = request.args.get("name", "world")
html = "<h1>Hello, " + str(escape(name)) + "!</h1>"
```

I ran all three payloads against it — `<script>alert(1)</script>`, `<ScRiPt>alert(1)</ScRiPt>`, and `<img src=x onerror=alert(1)>` — and every one came back as escaped text (`<script>...`), no alert firing. Worked on the first try. I think it worked because I named the exact fix I wanted

(`markupsafe.escape()`, contextual encoding) and specifically told it not to use a blacklist, instead of just asking it to "fix the XSS" and letting it pick whatever approach it wanted — which is how I got the broken blacklist fix earlier in this worksheet.